methods via define_method, method_missing, or something more esoteric, you need to ensure that your respond_to? method works in sync.

For partial test doubles, Mocha provides the method stubs, which is mixed in to any Ruby object:

minitest/01/test/models/project_with_stubs_test.rb

```
Line 1 test "let's stub an object" do
2   project = Project.new(name: "Project Greenlight")
3   project.stubs(:name)
4   assert_nil(project.name)
5 end
```

This test passes. Line 3 sets up the stub, which intercepts the project.name call in line 4 to return nil, and never even gets to the project name.

Mocha allows you to specify a return value for the stubbed method:

minitest/01/test/models/project_with_stubs_test.rb

```
test "let's stub an object again" do
  project = Project.new(name: "Project Greenlight")
  project.stubs(:name).returns("Fred")
  assert_equal("Fred", project.name)
end
```

The stubs method returns a Mocha Expectation object, which is effectively a proxy to the real object, but responds to a number of methods that let you annotate the stub. The returns method is one of those annotation messages that associates the return value with the method. Mocha also provides the shortcut project.stubs(name: "Fred"), allowing you to specify messages and return values as key/value pairs.

As with RSpec, you can stub classes as well as instances:

minitest/01/test/models/project_with_stubs_test.rb

```
test "let's stub a class" do
  Project.stubs(:find).returns(Project.new(name: "Project Greenlight"))
  project = Project.find(1)
  assert_equal("Project Greenlight", project.name)
end
```

And you can stub all instances of a class with the method any_instance, as in this example:

```
Project.any_instance.stubs(:save).returns(false)

stubby.stubs(:user_count).raises(Exception, "oops")
```